Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr.10

Disciplina: Tehnici de programare

Tema: ”Apărarea:detecția unui atac brute force din auth.log”

A efectuat : st. gr.SI –263 Boicu Ecaterina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea pune studentul în rolul apărătorului . Dintr-un jurnal auth.log de pe o mașină de laborator atacată , trebuie scris un program care :

Scoate robust eșecurile de autentificare și le numără pe adrese IP (Lucrarea A , parse_auth.py);

Decide ce adrese sunt suspecte , cu prag reglabil , validare la intrare și raport JSON (Lucrarea B , detector.py, test_detector.py).

Tehnicile exersate : funcții care întorc un rezultat (nu doar tipăresc), valori implicite pentru parametri , try/except, raise , expresii regulate , Counter , argparse, JSON , teste cu pytest.

I.

MEDIUL DE LUCRU ŞI PREGĂTIRE

ELEMENT	VALOARE
Sistem	macOS , terminal zsh integrat în VS Code
Python	3.9.6,rulat cu python3
Mediu Virtual	.venv (activ,vizibil în prompt ca (.venv))
Test runner	pytest 8.4.2
Depozit	Tehnici-de-programare-Proiect (GitHub), folder local ~/Documents/Tehnici de Programare

Fig.1- Tabelul pregătirii pentru laborator , mediul de lucru

Crearea structurii de foldere şi a pachetelor Python:

```
mkdir -p arsenal/aparare/start
touch arsenal/__init__.py
arsenal/aparare/__init__.py
python3 -m pip install pytest
```

Fig.1.2 - Comenzile de creare a folder-urilor

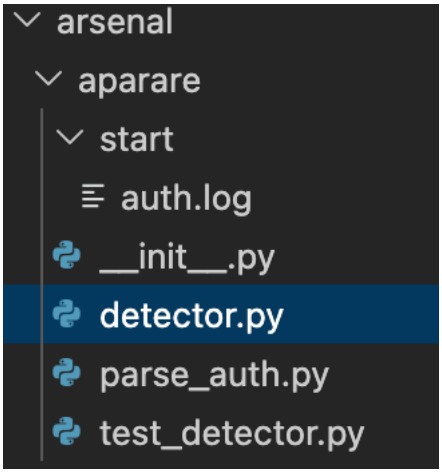


Fig.1.3 - Structura finală a lucrării

Fișierul de start auth.log

Fișierul auth.log nu era în Downnloads (comanda mv a dat : No such file or directory) , așa că l-am creat în arsenal/aparare/start/auth.log cu cat >…<<‘EOF’, cu conținutul exact al jurnalului din laborator.

```
wc -l arsenal/aparare/start/auth.log
```

Fig.1.4 - Verificare , rezultatul fiind “19”

Analiza conținutului (19 rânduri)

TIp de rând	Nr.
Failed password de la 203.0.113.9(atacatorul)	13
Failed password de la 198.51.100.7 (utilizator legitim,ana)	2
Accepted password/Accepted publickey	2
Rând CRON (sesiunea cron)	1
Rând stricat , fără structură (<linia stricata fara structura >)	1

Fig.1.5 - Analiza conținutului în formă de tabel

Deci, jurnalul conține 15 eșecuri de autentificare , 3 rânduri normale și un rând stricat pus intenționat , ca să testeze rezistența programului. Atacul brute force vine de la 203.0.113.9: 13 încercări în câteva minute , cu utilizatori diferiți (root,admin,test,oracle,git).

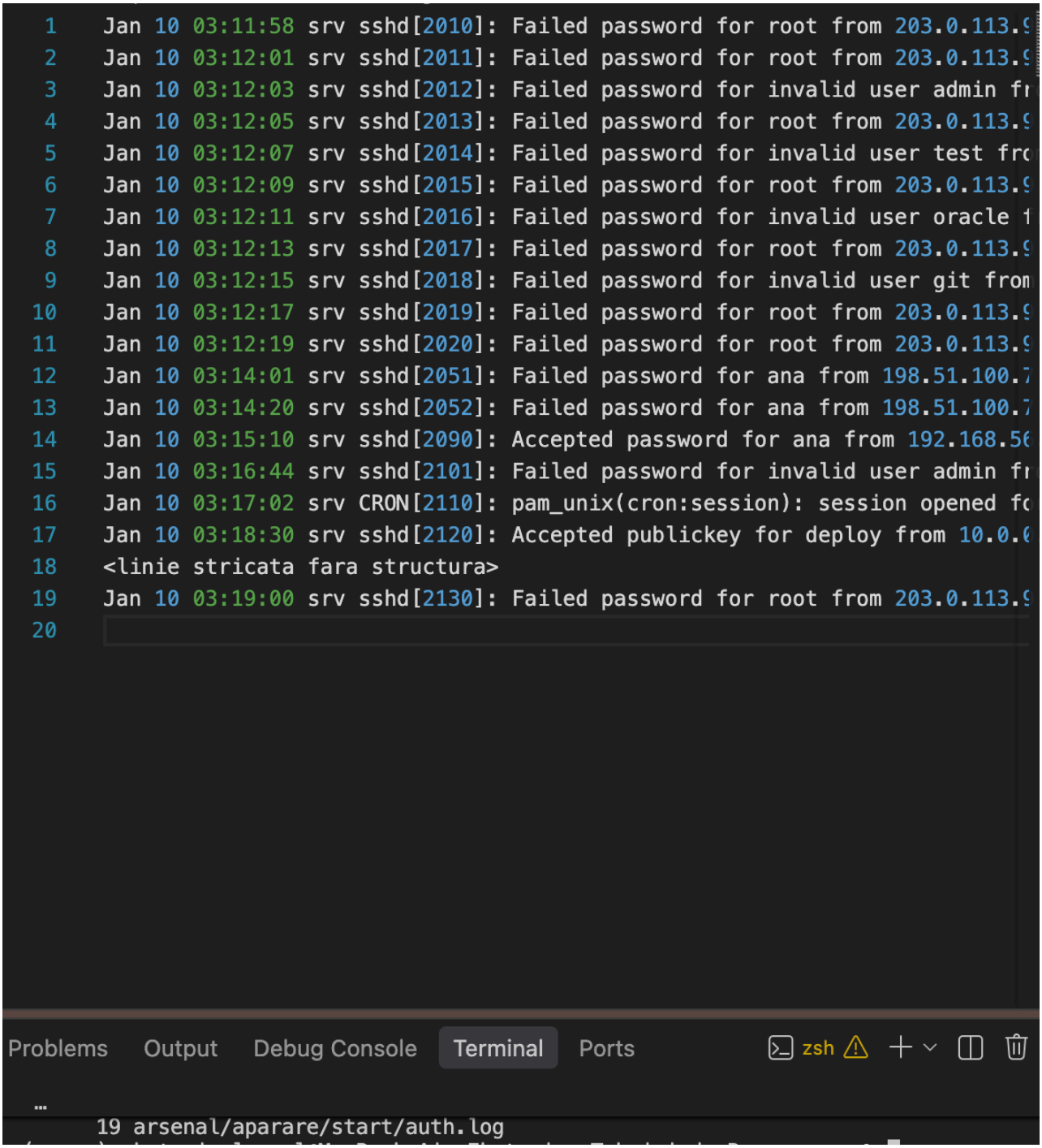


Fig.1.6 - Fișierul auth.log și rezultatul comenzii wc -l

II

LUCRAREA A.

parse_auth.py: scoate și numără eșecurile

OBIECTIV: arsenal/aparare/parse_auth.py citește jurnalul , scoate eșecurile de autentificare într-o listă de fișe și le numără pe adrese , fără să se oprească la un rând stricat.

A1. Funcția esec(linie),care întoarce o fișă - un rând de eșec arată așa: Failed password for root from 203.0.113.9 port 51230 ssh2. Funcția esec folosește expresia regulată dată:

```
import re
TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from
(\S+) port”)
```

```
def esec(linie):
    """Intoarce (user, ip) daca linia e un esec de autentificare, altfel
None."""
    m = TIPAR.search(linie)
    if m is None:
        return None
    return (m.group(1), m.group(2))
```

Fig.2.1 - Expresia introdusă

Cum funcționează?
`TIPAR.search(linie)` :caută structura în rând și întoarce un obiect Match sau None .

Grupul 1 (\S+) prinde utilizatorul , grupul 2 (\S+) prinde adresa IP.
Partea (?:invalid user)? E opțional și nu creează grup , deci rândurile de tip Failed password for invalid user admin from… dau tot utilizatorul admin , nu textul invalid .
Dacă rândrul nu e un eșec , funcția întoarce None. None nu e o eroare , e un semnal “aici nu e nimic de raportat” . Cel care cheama funcția decide ce face cu el .

```
python3 -c "from arsenal.aparare.parse_auth
import esec; print(esec('... Failed password for
root from 203.0.113.9 port 51230 ssh2'));
print(esec('<linie stricata fara structura>'))"
```

Fig.2.2 - Comanda de verificare

```
('root', '203.0.113.9')
None
```

Fig.2.3 - Rezultatul comandei A1

A2. Funcția citeste_esecuri(cale), robustă/rezistentă la rânduri stricate

```python
def citeste_esecuri(cale):
    """Intoarce lista de dictionare {"user": ..., "ip": ...} din
jurnal."""
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
        print("nu gasesc", cale)
        return esecuri
    with f:
        for linie in f:
            r = esec(linie)
            if r is not None:
                esecuri.append({"user": r[0], "ip": r[1]})
    return esecuri
```

Fig.2.4 - Codul A2

Cum funcționează?
Deschiderea fișierului e într-un try/except FileNotFoundError: un fișier lipsă dă mesajul clar nu gasesc<cale> și o listă goală, nu un traceback.
with f: închide fișierul automat la sfârșit.
Fiecare rând trece prin esec. Dacă rezultatul nu e None , se adaugă un dicționar {“user”:…,” ip”…}. Rândurile Accepted , CRON și rândul stricat dau None și sunt sărite , deci nu opresc parcurgerea.

```
python3 -c "
from arsenal.aparare.parse_auth import
citeste_esecuri
e =
citeste_esecuri('arsenal/aparare/start/auth.log'
)
print('esecuri:', len(e))"
```

Fig.2.5 - Comanda de verificare



Fig.2.6 - Rezultatul A2 + mesajul opțional

A3. Funcția pe_ip(esecuri), care numără pe adrese

```
from collections import Counter
…
def pe_ip(esecuri):
    """Intoarce un Counter al adreselor IP din lista de esecuri."""
    return Counter(e["ip"] for e in esecuri)
```

Fig.2.7 - Codul A3

Cum funcționează?
Counter din modulul collections numără de câte ori apare fiecare valoare. Funcția întoarce numărătoarea . Metoda most_common() o sortează de la mare la mic , deci atacatorul apare primul .

```
python3 -c "
from arsenal.aparare.parse_auth import
citeste_esecuri, pe_ip
e =
citeste_esecuri('arsenal/aparare/start/auth.log'
)
for ip, n in pe_ip(e).most_common():
    print(f'  {ip}: {n}')"
```

Fig.2.8 - Comanda de verificare

```
  203.0.113.9: 13
  198.51.100.7: 2
```

Fig.2.9 - Rezultatul A3

O adresă iese mult peste rest (13>2). Aceasta este semnătura unui atac brute force.

A4. Funcția main() și rezumatul

```
import sys
…
def main():
    if len(sys.argv) < 2:
        print("utilizare: python -m arsenal.aparare.parse_auth
<jurnal>")
        return
    esecuri = citeste_esecuri(sys.argv[1])
    counts = pe_ip(esecuri)
    print("Esecuri de autentificare:", len(esecuri))
    for ip, n in counts.most_common():
        print(f"  {ip}: {n}")


if __name__ == "__main__":
    main()
```

Fig.2.10 - Codul A4

Cum funcționează?

main() leagă funcțiile ca într-o conductă : citeste_esecuri produce lista , pe_ip o transformă în numărătoare , iar main tipărește rezumatul . Blocul if __name__= = “__main__”: face ca main() să ruleze doar când fișierul e pornit direct , nu când e importat (important pentru Lucrarea B , care-l importă)

```
python3 -m arsenal.aparare.parse_auth
arsenal/aparare/start/auth.log
```

Fig.2.11- Comanda de rulare , din rădăcina depozitului

```
esecuri: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

Fig.2.12 - Rezultatul final A4

III.

LUCRAREA B.

detector.py: detectorul de brute force

OBIECTIV: arsenal/aparare/detector.py decide ce adrese sunt suspecte , cu prag adaptabil , validare la intrare și raport JSON. Refolosește funcțiile din parse_auth printr-un import.

B1. Funcția suspecti(counts, prag=10)

```
def suspecti(counts, prag=10):
    """Intoarce lista de (ip, n) cu n >= prag, sortata descrescator dupa
n."""
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")
    return sorted(
        ((ip, n) for ip, n in counts.items() if n >= prag),
        key=lambda pereche: pereche[1],
        reverse=True,
    )
```

Fig.3.1 - Codul B1

Cum funcționează?
prag=10 , e o valoare implicită : dacă nu se transmite al doilea argument , pragul e 10 , dar se poate fi schimbat.
Generatorul păstrează doar perechile (ip,n) cu n>=prag.
sorted(…,key=lambda pereche: pereche[1], reverse=True) le ordonează descrescător după numărul de eșecuri.
Funcția întoarcere lista , nu o tipărește.

```
python3 -c "
from collections import Counter
from arsenal.aparare.detector import suspecti
c = Counter({'203.0.113.9': 13, '198.51.100.7':
2})
print(suspecti(c))
print(suspecti(c, prag=2))"
```

Fig.3.2 - Comanda de verificare

```
[('203.0.113.9', 13)]
[('203.0.113.9', 13), ('198.51.100.7', 2)]
```

Fig.3.3 - Rezultatul B1

Cu pragul implicit 10 , doar atacatorul trece . Cu prag 2 apare și a doua adresă. Pragul separă zgomotul (o parolă greșită) de semnal (un atac), de aceea e un parametru .

B2. Validarea cu raise

Un prag sub 1 nu are sens. Funcția îl oprește la intrare cu raise ValueError(…), în loc să continue și să marcheze orice adresă ca suspectă , în tăcere.

```
python3 -c "from arsenal.aparare.detector import
suspecti; suspecti({}, 0)"
```

Fig.3.4 - Comanda de verificare

```
rsenal.aparare.detector import suspecti; suspecti({}, 0)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "/Users/ekaterinalungul/Documents/Tehnici de Programare/arsenal/aparare/detector.
py", line 11, in suspecti
    raise ValueError(f"prag invalid: {prag} (cel putin 1)")
ValueError: prag invalid: 0 (cel putin 1)
```

Fig.3.5 - Rezultatul B2

Traceback-ul e așteptat aici, pentru că s-a cerut intenționat un prag absurd. Ultima linie , ValueError: prag invalid: 0 (cel putin 1), coincide cu fișa și arată că funcția a fost oprită la linia cu raise.

B3. argparse și raportul JSON

```
import argparse
import json
from collections import Counter

from arsenal.aparare.parse_auth import citeste_esecuri


def suspecti(counts, prag=10):
    """Intoarce lista de (ip, n) cu n >= prag, sortata descrescator dupa
n."""
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")
    return sorted(
        ((ip, n) for ip, n in counts.items() if n >= prag),
        key=lambda pereche: pereche[1],
        reverse=True,
    )


def main():
    p = argparse.ArgumentParser()
    p.add_argument("log")
    p.add_argument("--prag", type=int, default=10)
    p.add_argument("--json", action="store_true")
    a = p.parse_args()

    esecuri = citeste_esecuri(a.log)
```

```
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, a.prag)

    if a.json:
        raport = {
            "esecuri": len(esecuri),
            "pe_ip": dict(counts.most_common()),
            "suspecti": [{"ip": ip, "incercari": n} for ip, n in sus],
        }
        print(json.dumps(raport, indent=2))
    else:
        print("Esecuri de autentificare:", len(esecuri))
        for ip, n in counts.most_common():
            print(f"  {ip}: {n}")
        print(f"Suspecti (prag {a.prag}):")
        for ip, n in sus:
            print(f"  {ip}  ({n} incercari)  BRUTE FORCE")


if __name__ == "__main__":
    main()
```

Fig.3.6 - Codul B3 și întregii lucrării B

Cum funcționează?
log e un argument pozițional (calea jurnalului); —prag este un întreg cu valoarea implicită 10 (type=int); —json e un comutator / inversor (store_true).
citeste_esecuri este importat din parse_auth: refolosirea de cod prin import.
Fără —json , programul tipărește rezumatul citibil , cu BRUTE FORCE pe fiecare suspect.
Cu —json , tipărește cu json.dumps un raport cu cheile esecuri , pe_ip și suspecti.

```
python3 -m arsenal.aparare.detector
arsenal/aparare/start/auth.log
```

Fig.3.7 - Comanda de verificare

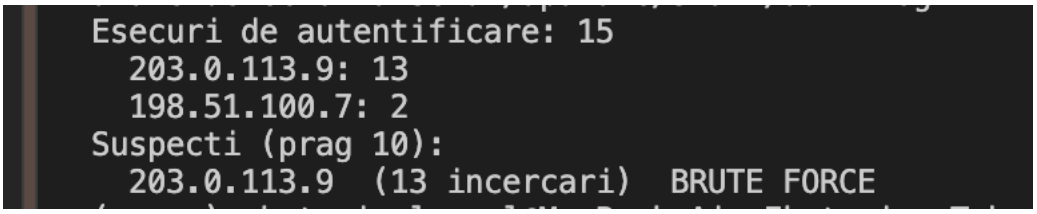


Fig.3.8 - Rezultatul B3

```
(.venv) ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % python3 -m arsenal
.aparare.detector arsenal/aparare/start/auth.log --json --prag 2
{
  "esecuri": 15,
  "pe_ip": {
    "203.0.113.9": 13,
    "198.51.100.7": 2
  },
  "suspecti": [
    {
      "ip": "203.0.113.9",
      "incercari": 13
    },
    {
      "ip": "198.51.100.7",
      "incercari": 2
    }
  ]
}
```

Fig.3.9 - Rularea și rezultatul raportul JSON

Cu —prag 2 , apar ambele adrese în lista de suspecți , ceea ce confirmă că, pragul se modifică din linia de comandă.

B4. Teste cu pytest

Fișierul arsenal/aparare/test_detector.py conține două teste cu date scrise de mână , deci rulează fără jurnal real:

```
from collections import Counter

from arsenal.aparare.detector import suspecti


def test_pragul_taie():
    c = Counter({"a": 13, "b": 2})
    assert suspecti(c, 10) == [("a", 13)]


def test_prag_invalid():
    import pytest
```

```
    with pytest.raises(ValueError):
        suspecti({}, 0)
```

Fig.3.10 - Codul B4

Cum funcționează?
`test_pragul_taie()`
verifică faptul că pragul 10 taie adresa b (2 eșecuri) și o păstrează doar pe a(13)

`test_prag_invalid():`
verifică faptul că pragul 0 ridică ValueError , folosind pytest.raises .

```
python3 -m pytest -q
arsenal/aparare/test_detector.py
```

Fig.3.11 - Comanda de verificare



Fig.3.12 - Rezultatul B4

CONCLUZII

În concluzie , la această lucrare , am învățat , pas cu pas , un detector de brute force , pornind de la un jurnal de autentificare . Am făcut cunoștință cu următoarele noțiuni:

Funcțiile care întorc rezultate: esec , citeste_esecuri , pe_ip și suspecti , întorc valori , nu tipăresc. Din acest motiv se leagă într-un canal (rând, fișă, listă, numărătoare, suspecți) și pot fi testate fără jurnal.

None ca semnal.esec întoarce None pentru un rând care nu e un eșec. Nu e o eroare , doar “nimic de raportat”, iar citeste_esecuri decide să-l sară.

Robustețe. Un rând stricat nu oprește parcurgerea , iar un fișier lipsă dă un mesaj clar datorită try/except FileNotFoundError.

Parametru cu valoare implicită . Pragul prag=10 , e o valoare implicită din suspecti , reglabilă prin —prag. El separă zgomotul (o parolă greșită) de semnal (un atac).

Validare cu raise . Un prag sub 1 oprește funcția la intrare cu ValueError și mesaj clar , în loc să marcheze orice adresă ca suspectă , în tăcere.

Rezultat. Atacatorul 203.0.113.9 a fost identificat cu 13 încercări , față de 2 pentru 198.51.100.7, adresa unui utilizator legitim. Doar prima depășește pragul 10 .

Lucrarea m-a ajutat să înțleg și partea de apărare a aceleiași tehnici folosite de atacator : ce lasă în urmă un brute force într-un jurnal și cum se detectează automat.